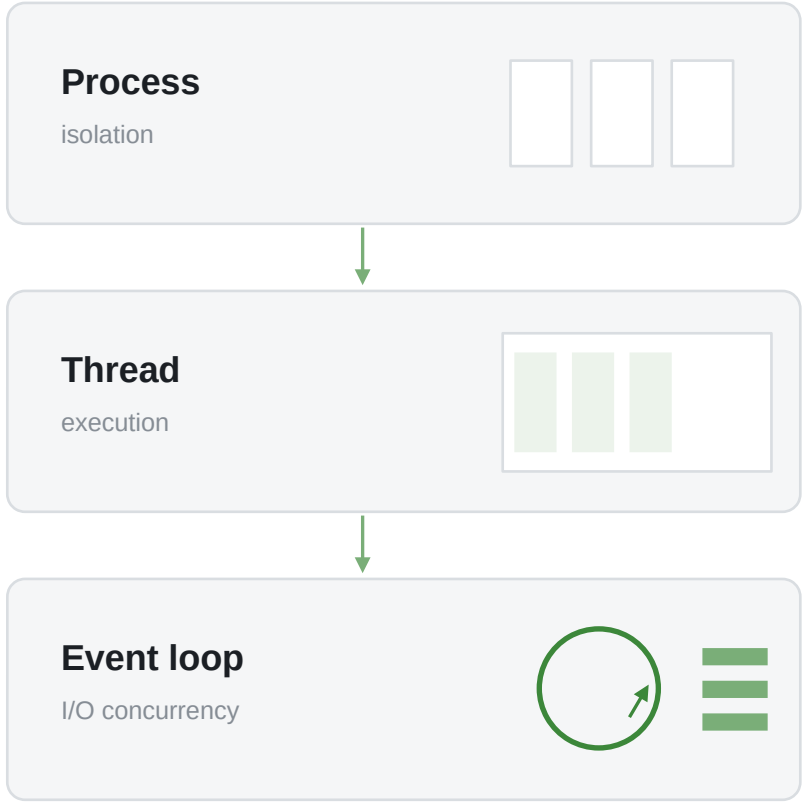
ELEVATE · WEEK 1 · SESSION 1

Node.js Under the Hood

Processes, threads, and the road to the event loop

Part 1 of 2

What the operating system gives us, and how web servers learned to handle many connections at once — before we look inside Node.js itself.



Foundations → concurrency models → event-driven I/O

AGENDA

What we'll cover

TODAY — FOUNDATIONS

01 **Process**
The operating system's unit of isolation

02 **Thread**
The operating system's unit of execution

03 **Server concurrency**
Four architectural eras, and their trade-offs

Two parts: the operating system's primitives first, then the Node.js runtime built on top of them.

NEXT — NODE.JS INTERNALS

04 **Node.js architecture**
One process, one main thread, one event loop

05 **Event loop deep dive**
Phases, queues, and starvation

06 **CPU-bound vs I/O-bound**
Where the time actually goes

07 **Blocking, libuv pool, workers**
Getting parallelism back safely

What is a process?

> **A running instance of a program — with its own memory, resources, and identity.**

Program ≠ process

The binary on disk becomes a process when the OS loads it and starts executing.

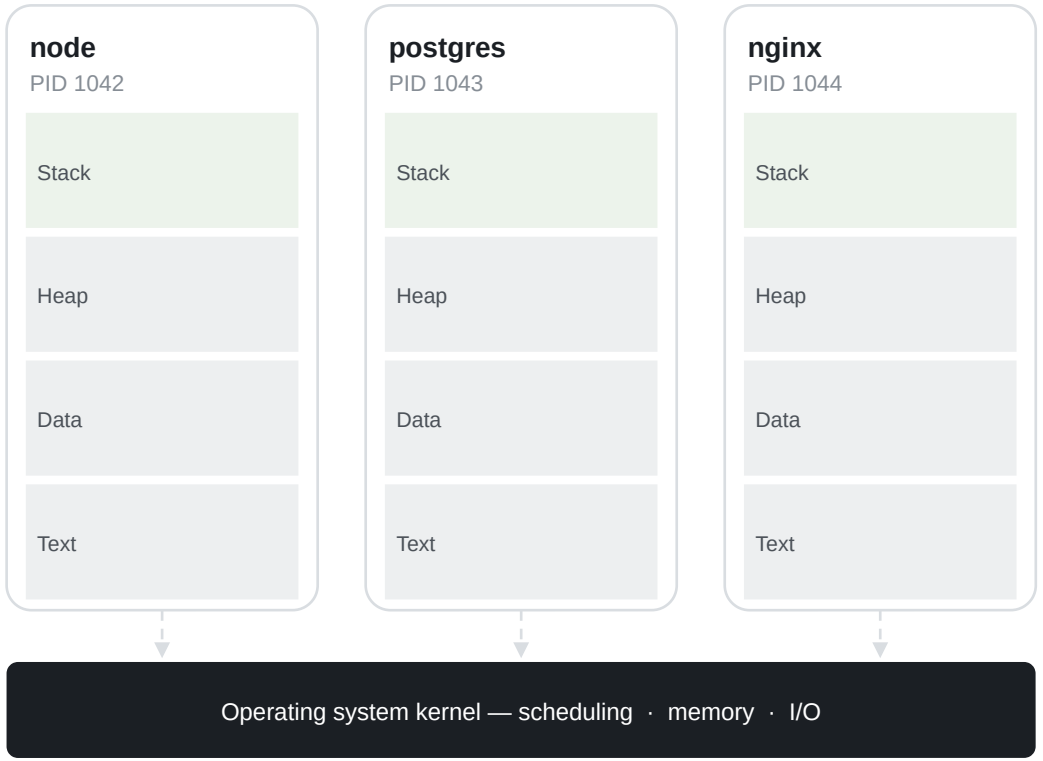
Private address space

Another process cannot read or write its memory directly. Sharing means IPC — pipes, sockets, shared memory.

Own identity

A PID, file descriptors, credentials, signal handlers, and scheduling state.

Three processes, three private address spaces — communication needs IPC



Anatomy of a process

Virtual address space

Stack
call frames, local variables
grows down

Heap
dynamically allocated objects
grows up

Data / BSS
globals and static data

Text
machine code (shared from binary)

Process Control Block
the kernel's record

- PID
- State
- Program counter
- Registers
- Page table
- Open files

WHAT MATTERS FOR US

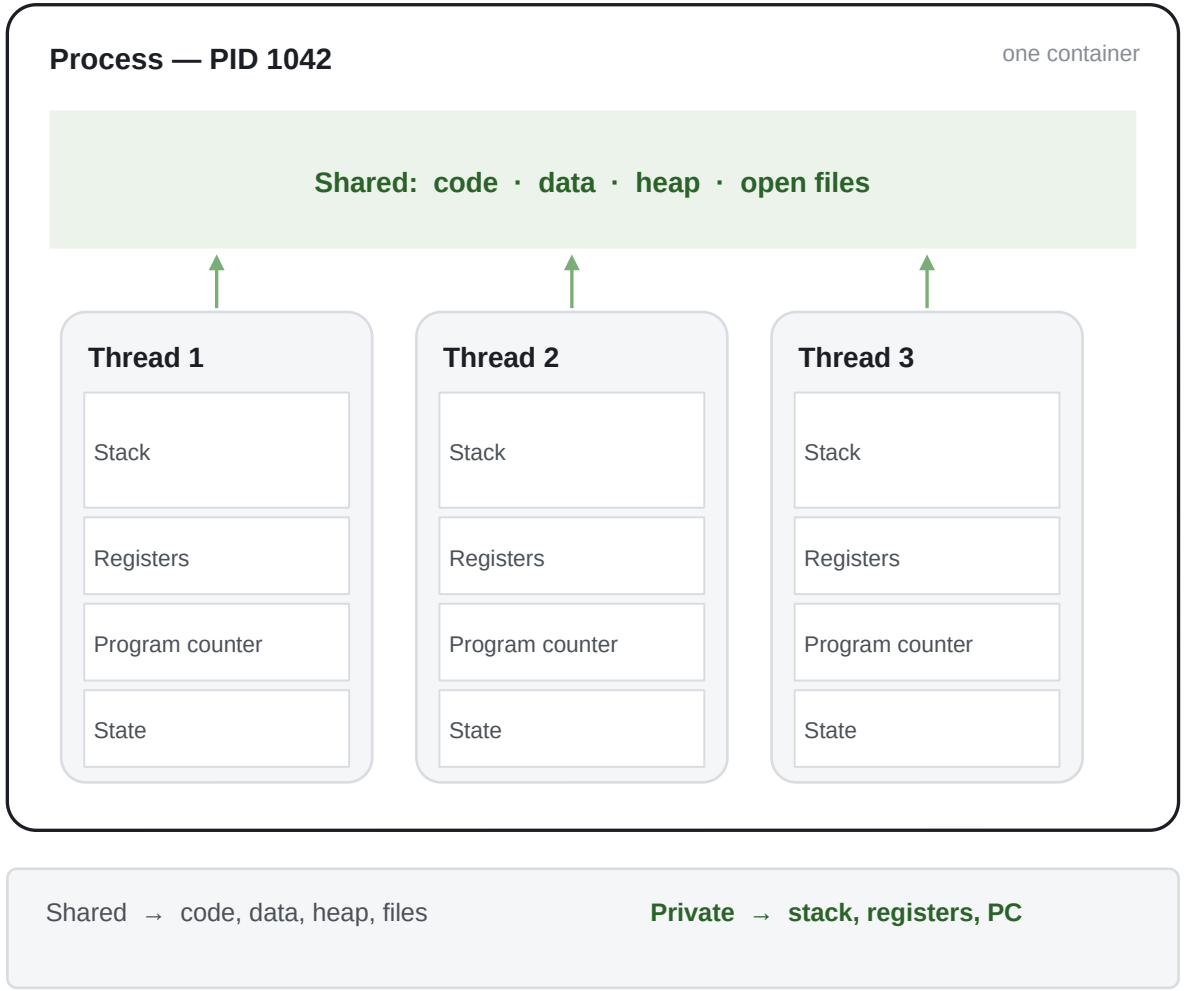
- Text is shared from the binary; data, heap, and stack belong to the process.
- The stack is per thread. The heap is shared by all threads in the process.
- The PCB is the state the kernel saves and restores when it switches processes.
- A context switch swaps address spaces — that is the expensive part.

New process = new address space + new PCB.

What is a thread?

The smallest unit of execution the OS schedules. A thread lives inside a process and shares its memory.

- Every process starts with one thread — the main thread.
- Each thread has its own stack, registers, and program counter.
- Threads share code, data, heap, and open files — cheap communication, but no isolation.
- The OS can run threads on different cores, or interleave them on one.
- A thread is the thing the scheduler actually runs; the process is the container.



Process vs thread

Criterion	Process	Thread
Memory	Private address space	Shared process memory
Creation cost	Heavy — new address space and PCB	Light — stack and registers only
Communication	IPC: pipes, sockets, shared memory	Direct reads and writes to shared memory
Failure isolation	A crash is contained	One bad thread can take down the process
Context switch	Expensive — swap page tables	Cheaper — same address space

Threads are cheaper than processes — but they are not free, and they are not isolated.

Concurrency is not parallelism

Concurrency — one core

Interleaved progress, many tasks in flight



time → A, B and C are three tasks; ticks mark context switches

Progress is real, but only one task executes at any instant. Slow I/O is hidden by switching.

Parallelism — two cores

Simultaneous execution on separate hardware



Both tasks run at the same instant. Parallelism requires more than one core.

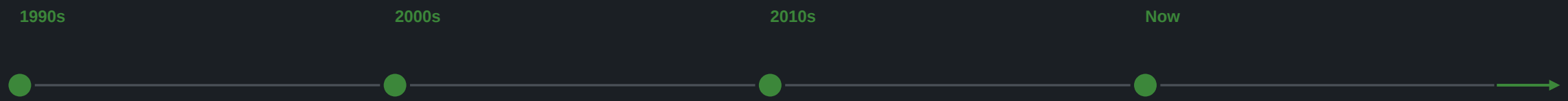
Concurrency
Dealing with many things at once — a structural property.

Parallelism
Doing many things at once — requires multiple cores.

Node.js is concurrent by design. Parallelism is something you have to add deliberately.

How server concurrency evolved

Four architectures, one goal: serve more connections with fewer resources per connection.



Process per connection

fork a process for every client

Thread per connection

one thread per connection, shared memory

Event-driven non-blocking I/O

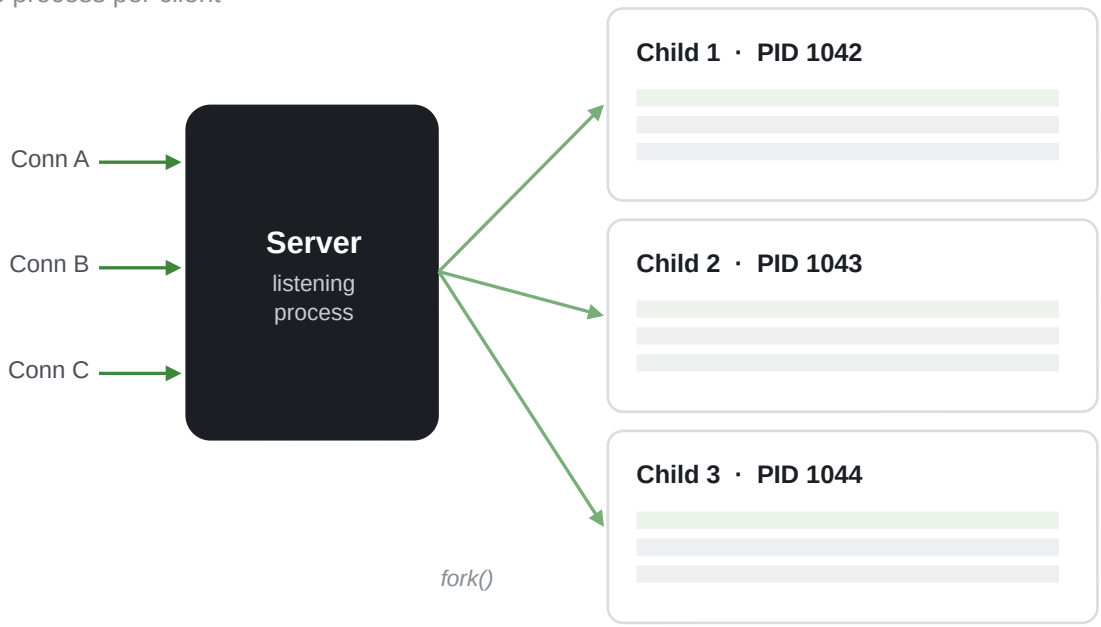
register interest, react to ready events

Hybrid event loops + pools

loops on every core, pools for blocking work

Process per request / connection

One process per client



Each child gets its own copy of the address space.

HOW IT WORKS

1. The listener accepts a connection.
2. It forks a child process for that connection.
3. The child handles the request and exits.

STRENGTHS

- Strong isolation between clients
- A crash stays contained
- Simple, blocking code

COSTS

- fork / exec is expensive
- Context switches cost more
- Many processes need real memory

Fine for a handful of clients. Collapses under thousands of concurrent connections.

Thread per request / connection

MULTITHREADED SERVERS

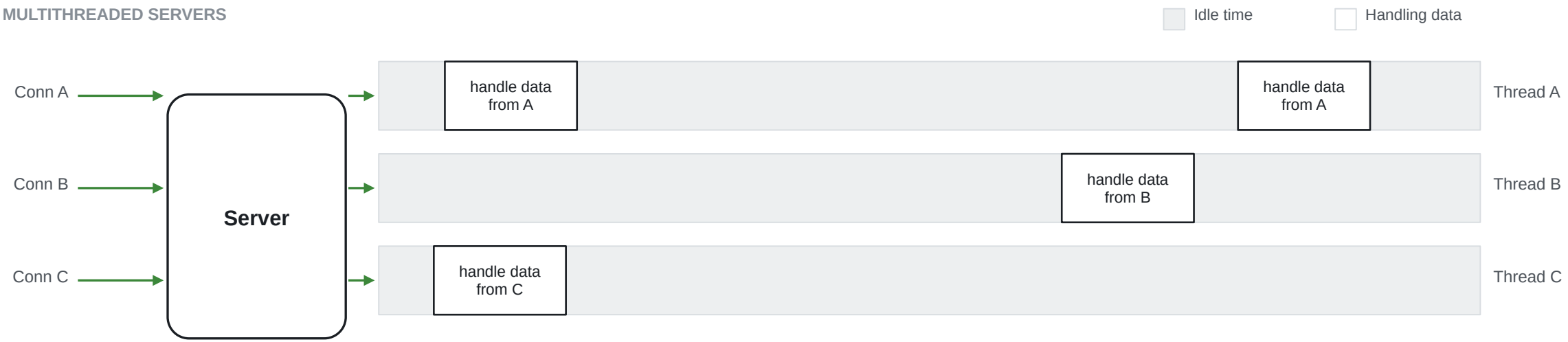


Figure 1.1 — Using multiple threads to process multiple connections

WHY IT FELT BETTER

- Blocking code is straightforward to write.
- A thread can wait on a database or network response while others work.
- No manual async plumbing; the runtime handles the waiting.

WHERE IT BREAKS

- Large thread counts mean memory and scheduling overhead.
- Every thread reserves a stack, even while idle.
- Context switching grows with concurrency, not with useful work.

Why thread-per-request hits a wall

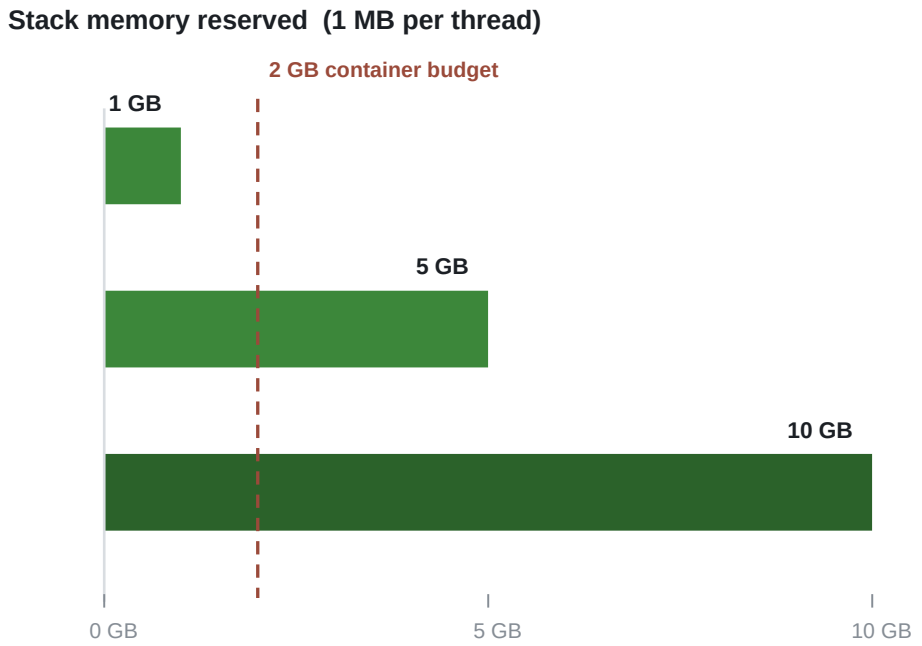
10,000
concurrent connections

× 1 MB default stack

≈ 10 GB of stacks

reserved before a single request is processed

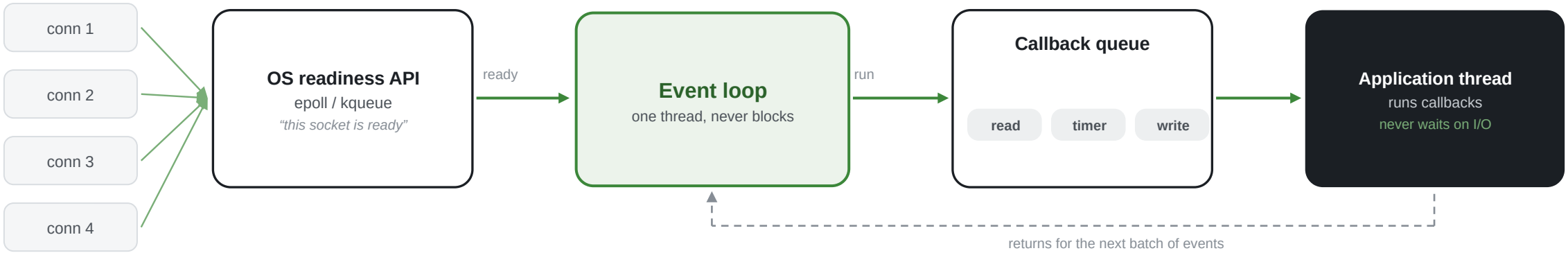
- Most threads sit idle, waiting on I/O.
- The scheduler pays to switch between them.
- Cost grows with connections, not with work.



The limit isn't concurrency. It's dedicating a thread to every connection that is mostly waiting.

Event-driven, non-blocking I/O

ONE THREAD, MANY CONNECTIONS



WHY IT WINS

- One thread manages thousands of connections.
- Per-connection memory is a socket and a small state object.
- Excellent for I/O-heavy workloads — the common web case.

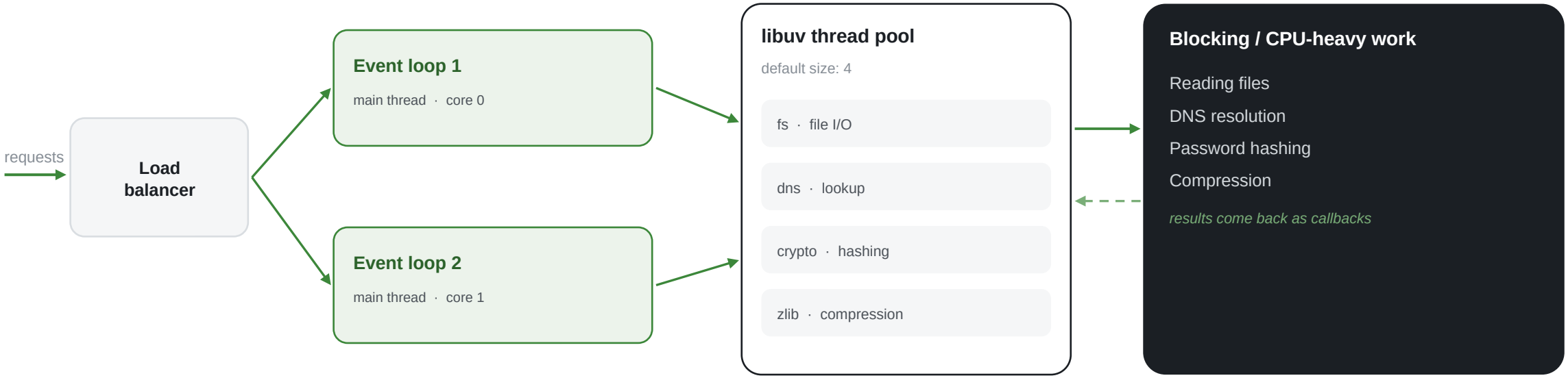
WHAT IT DEMANDS

- Every operation must be non-blocking.
- Control flow becomes asynchronous — callbacks, promises, async/await.
- One slow callback stalls every other connection.

This is the model Node.js is built on — and the source of its most famous failure mode.

Hybrid architectures

MODERN SERVERS



WHAT IT BUYS

- Efficient I/O concurrency with real parallel execution.
- Multiple event-loop processes can use every CPU core.
- Blocking and CPU-heavy work moves off the loop.

WHAT IT COSTS

- Shared resources need coordination: pools, caches, queues.
- Work distribution and back-pressure become your problem.
- Process count and pool size become real tuning knobs.

Each event loop is pinned to a core; the pool is per process. Node.js is this model.

WRAP-UP

What to remember from Part 1

01

A process is isolation.

Its own memory, PID, and resources. Creating one is expensive, and communication means IPC.

02

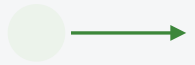
A thread is execution.

Shared memory, private stack and registers. Cheaper than a process, but every thread still costs.

03

Concurrency is an architecture choice.

Process → thread → event loop → hybrid. Each step removes per-connection overhead and adds new complexity.



NEXT · PART 2

Node.js architecture and the event loop in depth

One process, one main thread, one event loop — and the thread pool working behind it.



Questions before we open the hood?